

SOFTWARE REQUIREMENTS SPECIFICATION

Hotel Management System

Object-Oriented Design (OOD) — Full Requirements Document



Prepared by: Nonceba
Module: WeThinkCode_ — Java OOP
Document type: System Requirements Specification (SRS)
Date: 14 July 2026

SECTION 01

1. Introduction

1.1 Purpose

This document defines the full set of requirements for the **Hotel Management System (HMS)**, an object-oriented software system designed to manage hotel operations including room bookings, guest check-in/check-out, housekeeping, billing, and guest notifications. It is intended to guide the object-oriented design (OOD) and implementation of the system, and to serve as the reference requirements document for the assessment / project deliverable.

1.2 Scope

The Hotel Management System will allow guests to search for and book rooms, allow receptionists to manage bookings and the check-in / check-out process, allow housekeeping staff to track room cleanliness and maintenance status, allow managers to oversee staff and room inventory, and allow the system itself to automatically issue notifications for bookings, cancellations, and billing events. The system covers a single hotel property with multiple room types, and models the core domain objects (Hotel, Room, RoomBooking, RoomKey, Housekeeping, RoomCharge, Invoice, and Notification) using object-oriented principles.

1.3 Intended Audience

- Developers implementing the class structure and business logic
- Assessors / lecturers reviewing the OOD breakdown
- Project teammates contributing to related modules

1.4 Definitions & Abbreviations

Term	Definition
HMS	Hotel Management System
OOD	Object-Oriented Design
Actor	A user or external system that interacts with the HMS
RoomKey	A digital or physical key record (barcode / key ID) assigned to a room for guest access
FIFO	First-In-First-Out — used where queued housekeeping or charge processing applies

SECTION 02

2. Functional Requirements

The table below expands the core system requirements into a traceable, numbered list of functional requirements (FR), grouped by capability area.

FR	Requirement	Area
FR-01	The system shall support multiple room types: Standard, Deluxe, Family, and Business Suite, each with distinct pricing and attributes.	Room Management
FR-02	The system shall allow guests to search available rooms by date range, room type, and price.	Booking
FR-03	The system shall allow guests to make a room booking, specifying check-in and check-out dates.	Booking
FR-04	The system shall allow guests to cancel an existing booking.	Booking
FR-05	The system shall allow receptionists to create bookings on behalf of a guest (walk-in or phone booking).	Front Desk
FR-06	The system shall allow receptionists to check a guest in, verify the booking, and issue a RoomKey.	Front Desk
FR-07	The system shall allow receptionists to check a guest out and close the associated booking.	Front Desk
FR-08	The system shall allow receptionists to add or modify room records (availability, price, status).	Room Management
FR-09	The system shall maintain a housekeeping log per room, recording cleaning and maintenance activity.	Housekeeping
FR-10	The system shall allow housekeeping staff to update a room's cleanliness / maintenance status.	Housekeeping
FR-11	The system shall allow guests to request room service (RoomCharge) during their stay.	Billing
FR-12	The system shall generate an invoice that itemises all charges against a room / booking.	Billing
FR-13	The system shall allow guests to pay bills via credit card, cheque, or cash.	Billing
FR-14	The system shall automatically send notifications to guests for booking confirmation, cancellation, and billing events.	Notifications
FR-15	The system shall allow managers to add new staff members (receptionists, housekeepers) to the system.	Staff Management

FR	Requirement	Area
FR-16	The system shall allow managers to manage and oversee staff assignments and schedules.	Staff Management
FR-17	The system shall assign a unique RoomKey (barcode / key ID) to each room and track which room it is currently assigned to.	Access Control

Table 2.1 — Functional requirements traceability list

SECTION 03

3. Actors & Use Cases

The system involves five main actors, each with a distinct set of responsibilities and interactions:

Guest	Search rooms, Make booking, Cancel booking, Request room service, Pay bill
Receptionist	Add / modify rooms, Check guests in / out, Create bookings, Process payments
Manager	Add new staff, Manage staff, View reports / occupancy
Housekeeper	Update housekeeping logs, Perform room maintenance
System	Send notifications, Trigger booking / cancellation alerts, Generate invoices

Table 3.1 — Main actors and their core responsibilities

3.2 Use Case Descriptions

UC-01 — Search & Book Room (Actor: Guest)

Description: Guest searches available rooms by date/type/price, selects a room, and confirms a booking.

Precondition: Rooms exist in the system with a status of 'Available'.

Postcondition: A RoomBooking is created and a confirmation notification is sent.

UC-02 — Check-In Guest (Actor: Receptionist)

Description: Receptionist verifies an existing booking on guest arrival and assigns a RoomKey, updating room status to 'Occupied'.

Precondition: A valid RoomBooking exists for the guest.

Postcondition: Guest receives a RoomKey; room status becomes 'Occupied'.

UC-03 — Cancel Booking (Actor: Guest / Receptionist)

Description: An existing booking is cancelled and its status updated; a cancellation notification is sent.

Precondition: A RoomBooking exists and has not yet been checked in.

Postcondition: Booking status is set to 'Cancelled'; room becomes available again.

UC-04 — Log Housekeeping Activity (Actor: Housekeeper)

Description: Housekeeper records cleaning / maintenance activity against a specific room.

Precondition: Room exists in the system.

Postcondition: A HouseKeeping record is created / updated for the room.

UC-05 — Request Room Service (Actor: Guest)

Description: Guest requests a service (e.g. food, laundry) which is logged as a RoomCharge against their room.

Precondition: Guest is currently checked in.

Postcondition: A RoomCharge entry is added and later appears on the Invoice.

UC-06 — Pay Bill (Actor: Guest / Receptionist)

Description: Guest settles their invoice via credit card, cheque, or cash at check-out or during their stay.

Precondition: An Invoice exists with one or more RoomCharge / booking line items.

Postcondition: Invoice is marked as paid; a payment notification may be sent.

UC-07 — Manage Staff (Actor: Manager)

Description: Manager adds new staff members and manages/assigns existing staff.

Precondition: Manager is authenticated with appropriate permissions.

Postcondition: Staff record is created, updated, or reassigned.

SECTION 04

4. Key Classes (Class Diagram Overview)

The domain model is composed of eight key classes. Each class encapsulates its own state and behaviour, following the OOP pillars of **encapsulation**, **abstraction**, **inheritance**, and **polymorphism**.

Class	Key Attributes	Responsibility
Hotel	name, address, list<Room>	Root aggregate; owns the collection of rooms for the property.
Room	number, style, status, price, RoomKey	Represents a single room; tracks type, price, and current status (Available / Occupied / Maintenance).
RoomBooking	checkInDate, checkOutDate, bookingStatus	Represents a reservation linking a Guest to a Room over a date range.
RoomKey	barcode, keyId, assignedRoom	Represents the access credential issued to a guest for a specific room.
HouseKeeping	roomId, logEntries, lastCleaned	Maintains the housekeeping/maintenance history for a room.
RoomCharge	chargeId, description, amount	Represents a single service or charge requested by a guest against a room.
Invoice	invoiceId, lineItems, totalAmount, paymentStatus	Aggregates RoomCharge and booking costs into a payable bill.
Notification	recipient, message, type, timestamp	Represents an alert sent to a guest (booking, cancellation, billing).

Table 4.1 — Key domain classes and their attributes

4.2 Key Relationships

- **Hotel 1 — * Room**: a Hotel owns many Rooms (composition).
- **Room 1 — 1 RoomKey**: each Room has an associated RoomKey when occupied.
- **Room 1 — * RoomBooking**: a Room can have many bookings over time.
- **RoomBooking 1 — * RoomCharge**: charges accumulate against an active booking.
- **RoomBooking / RoomCharge * — 1 Invoice**: charges and booking costs roll up into one Invoice.
- **Room 1 — * HouseKeeping**: a Room accumulates a history of housekeeping log entries.
- **System — Notification**: the System class raises Notification instances on key events.

4.3 Applied OOP Principles

Principle	Application in this system
Encapsulation	Room, RoomBooking, and Invoice expose controlled getters/setters; internal state (e.g. booking status transitions) is only modified through class methods, not directly.
Abstraction	A Staff abstract base class can define common behaviour for Receptionist, Manager, and Housekeeper, hiding role-specific implementation details behind a shared interface.
Inheritance	Receptionist, Manager, and Housekeeper can extend a common Staff class, inheriting shared attributes such as staffId and name.
Polymorphism	A performDuty() method can be overridden differently by each Staff subclass (e.g. Receptionist checks guests in, Housekeeper logs cleaning), while being invoked the same way by the system.

Table 4.3 — Mapping of OOP pillars to the design

SECTION 05

5. Process Flows (Activity Diagrams)

Three core workflows drive the system's behaviour. Each is expressed below as a numbered activity sequence.

Room Booking

1. Search Rooms
2. Select Room
3. Confirm Booking
4. Send Notification

Check-In

1. Guest Arrives
2. Verify Booking
3. Assign Room Key
4. Status: Occupied

Cancel Booking

1. Request Cancel
2. Update Status
3. Send Cancellation

SECTION 06

6. Non-Functional Requirements

Category	Requirement
Performance	Room availability search results shall return within 2 seconds for a hotel with up to 500 rooms.
Usability	The interface shall allow a guest to complete a booking in no more than 4 steps.
Reliability	The system shall not allow double-booking of the same room for overlapping date ranges.
Security	Payment details shall be handled through a secure, PCI-aware payment flow; staff actions shall be role-restricted.
Scalability	The system shall be designed so additional room types, staff roles, or hotel branches can be added without restructuring core classes.
Data Integrity	Booking, invoice, and housekeeping records shall be persisted consistently even if a request partially fails (transactional updates).
Maintainability	Classes shall follow single-responsibility and low-coupling principles to support future extension (e.g. loyalty programs, multi-hotel support).

Table 6.1 — Non-functional requirements

SECTION 07

7. Assumptions & Constraints

7.1 Assumptions

- The system manages a single hotel property (multi-branch support is a future extension).
- Guests are assumed to have a unique identifier (e.g. email) for booking and notification purposes.
- Room status is limited to a defined set: Available, Occupied, Maintenance, Cleaning.
- Payments are processed through an external/simulated payment gateway; the system itself does not store raw card data.

7.2 Constraints

- The design must be implemented using Java, following core OOP principles (encapsulation, inheritance, abstraction, polymorphism).
- The system must support the four defined room types without hard-coding logic that would prevent future room types being added.
- Notifications must be triggered automatically by the System on booking, cancellation, and billing events — not manually initiated.

7.3 Out of Scope

- Multi-hotel / franchise management across branches.
- Third-party travel agency integrations (e.g. Booking.com, Expedia).
- Loyalty / rewards points program.

SECTION 08

8. Requirements Summary Checklist

A consolidated checklist mapping the original system requirements to their expanded functional requirement IDs, for quick reference during implementation and review.

Original Requirement	Mapped FR IDs
Support room types: Standard, Deluxe, Family & Business Suite	FR-01
Guests search rooms and make bookings	FR-02, FR-03
Receptionists create bookings, check-in & check-out	FR-05, FR-06, FR-07
Maintain housekeeping logs for rooms	FR-09, FR-10
Pay bills via credit card, check, or cash	FR-11, FR-12, FR-13
Send notifications for booking & cancellation	FR-14

Table 8.1 — Traceability back to the original slide requirements

End of Document

This SRS is ready to guide UML class diagram creation, sequence diagrams, and Java implementation for the Hotel Management System OOD project.